

Human Trust Based Federated Publication

Text Distribution Without Global Identity

Registry: [[HOWL-COMP-15-2026](#)]

DOI: 10.5281/zenodo.21901397

Date: August 2026

Domain: Distributed Systems / Federated Content Distribution / Network Protocol Design

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude Opus 5.0.

Abstract

Current federated publication systems bind a person to a global identity, carry a record of the path an item travelled, and require a directory or a registry for a server to participate. Each of these properties has a cost. A global identity makes a writer addressable across every server. A path record makes the first server of an item identifiable. A directory makes participation a thing that a third party can grant and withdraw.

This paper describes Human Trust Based Federated Publication, a protocol in which none of the three properties is present. A link between two servers is made by two people who exchange public keys outside the program. There is no directory and no registry. An item carries the writer handle, a topic string, a subject, a body, and a creation time, and carries nothing else. Loop control uses a hash that each server computes for itself from those fields and never transmits. A server that receives an item and publishes it again offers it onward as its own publication with a new local sequence number, so no path record is needed and none exists.

The result is a network in which the unit of trust is a pair of administrators, the unit of routing is a topic string that nobody owns, and the unit of storage is a fixed window that each server sets for itself. Growth in the number of servers makes the network more selective rather than larger, because each administrator accepts only the topics that the members of that server read.

The protocol requires one HTTP endpoint, one hash construction, one signature construction, one topic grammar, and one integer sequence. This paper gives the complete wire specification, the conformance requirements, and the test vectors necessary for an independent implementation.

Keywords: federated publication, distributed systems, trust, anonymity, content distribution, protocol design, Usenet, decentralisation

Contents

1. Introduction
2. Related work and the position of this design
3. The model
4. The three absent properties
5. The wire protocol
6. Replies
7. Control and its limits
8. Scale
9. Conformance
10. Properties and limits
11. Conclusion
 - Appendix A — Test vectors
 - Appendix B — Field travel
 - Appendix C — Failure modes
 - Appendix D — Deployment note

1. Introduction

1.1 The situation

Text publication at present occurs mostly on platforms that hold one global account for each person. The account is the unit of the system. The operator of the platform holds the account, the ranking of what a reader sees, the policy that governs what may be written, and the record of everything that was written.

A person who writes on such a platform accepts four conditions at once. The account can be closed. The ranking can change without notice. The policy is written by one organisation for every user of that organisation. The record is held by a party that the writer does not control.

These conditions are not defects of one operator. They follow from the shape of the system. A system with one global account for each person must hold those accounts somewhere, and the party that holds them has all four powers whether it wants them or not.

1.2 The existing federated alternatives

Federated systems remove the single operator. A person holds an account on one server among many, and servers exchange content.

This removes one problem and keeps two. In the common federated designs a writer still has a global identity, which resolves to a server and is addressable from every other server. An item still carries enough information to identify where it started. A server still usually needs to appear in some list, on some relay, or in some discovery service to be reachable at all.

The result is that the four powers of section 1.1 are divided among many operators rather than removed. This is an improvement, and it is not the same as their absence.

1.3 The three properties this design removes

This design removes three properties that the systems above hold in common.

A global identity of a writer. A handle exists on one server. It has no meaning on any other server. Two servers may hold a member with the same handle, and those are two different people.

Provenance of an item. An item carries no path, no hop count, and no server identity. A receiver cannot determine which server an item started on.

A directory or a registry. A server participates by speaking the protocol to a peer that agreed to speak it. Nothing else grants participation and nothing can withdraw it.

1.4 The property that replaces all three

One property replaces them. **A link between two servers is a relationship between two people.** Two administrators exchange public keys by any means outside the program, and each writes the key of the other into its own records. Every other decision in the system is local to one server or to one pair of servers.

This is the meaning of the phrase *human trust based* in the title. The trust is between two named people who chose each other. It is not delegated to a certificate authority, a registry, a reputation score, or a moderation service. It does not extend beyond the pair, and it does not transfer.

1.5 What this paper gives

This paper gives the complete specification necessary for an independent implementation:

- The wire protocol (section 5)
- The optional reply mechanism (section 6)
- The conformance requirements (section 9)
- The test vectors (Appendix A)

An implementation that follows sections 5 and 9 interoperates with any other implementation that follows them, in any programming language, on any storage system, with no relationship to any prior implementation and no permission from anybody.

1.6 What this paper does not give

This paper does not give a performance measurement, a deployment report, or a study of the behaviour of users. No large deployment of this protocol exists at the time of writing. The figures in section 8 are estimates derived from the design and from the stated field limits, and section 8 marks them as estimates.

This paper also does not argue that this design is better than the alternatives in section 1.2 for every purpose. It makes a different trade. Section 4 states the cost of each of the three absent properties in the same subsection as the benefit.

2. Related work and the position of this design

2.1 Comparison

Table 2.1 — Comparison with prior and current systems

Property	Usenet	Fidonet echomail	Private NNTP	RSS or Atom pull	ActivityPub systems	This design
Injection cost	Any feed, no relationship	Admission by a coordinator	Account on a server	Control of one site	An account on any instance	One administrator approves one pair
Global directory	The hierarchy and the newgroup process	The nodelist and the coordinators	None	None	Instance lists, relays, discovery services	None
Message identity	Message-ID, global	MSGID, global	Message-ID, global	A GUID or a URI	An HTTP URI, global and resolvable	None. A local hash only.

Property	Usenet	Fidonet echomail	Private NNTP	RSS or Atom pull	ActivityPub systems	This design
Author identity	An address in the header	A name and a node number	An address in the header	A name in the feed	An actor URI, global and re-solvable	A handle, local to one server, not unique
Loop control	The Path header	SEEN-BY and PATH lines	The Path header	Not applicable, no relay	Deduplication on the object URI	Re-origination and a local hash
Path visible to a reader	Yes	Yes	Yes	Not applicable	Partly, through the actor URI	No
Forwarding	Flood fill to all peers	Flood fill inside an echo	Flood fill to peers	None, the reader pulls	Push to followers and relays	Pull, and republication by choice
Transfer state	None, push of new articles	None, packet based	None	The reader holds a position	None, push for each activity	One integer for each pair
Retraction	Cancel messages, forgeable	Local only	Cancel messages	Remove from the feed	A delete activity, advisory	Local only, and stated as such

2.2 Usenet

Usenet is the closest prior system in shape. It has a dotted topic tree, a pull transfer between peers, and no central store.

Two properties of Usenet are not present here. Injection was open: any server that held a feed could inject an article, and no relationship with the receiving operator was required. Each article carried a Path header that recorded the servers it passed through, which was readable by any reader.

The literature on the decline of the open Usenet text hierarchies is informal and this paper does not cite a single authoritative account of it. The mechanism most often described is that injection at no cost and with no relationship permitted unsolicited bulk posting at a volume that the moderation tools of the period could not match. This paper treats that account as a design input and not as an established finding.

This design keeps the topic tree and the pull transfer. It removes open injection and it

removes the path record.

2.3 Fidonet echomail

Fidonet distributed message areas between nodes by scheduled transfer. Loop control used SEEN-BY and PATH lines in each message, which named the nodes that had already received it.

The relevant property for this paper is the nodelist. A node had a number, the number came from a coordinator, and the list of nodes was compiled and distributed. That is a directory. A directory is a thing that can grant participation and can withdraw it.

This design has no equivalent. The removal is deliberate and section 4.3 states its cost.

2.4 ActivityPub systems

The current federated publication systems in wide use are based on ActivityPub or on similar designs. An actor is identified by an HTTP URI. An object is identified by an HTTP URI. Both resolve, and both name a server.

This paper does not present that as a defect. The resolvable identity gives real capabilities that this design does not have: a reply that reaches its parent reliably, a mention that reaches a named person, a follow relationship that a person controls for themselves, and a delete that propagates as an instruction to a known set of recipients.

The cost of the resolvable identity is that the origin of every item is visible and every writer is addressable from every server. This design makes the opposite choice on both points and accepts the loss of the four capabilities above.

The choice is not a claim that one is correct. It is a claim that the two sets of properties are not compatible, which section 4.4 states in full.

2.5 Two design points that follow

Re-origination replaces path tracking. Every hop in this design is an ordinary local publication. A server that receives an item and publishes it again gives it a new local sequence number and offers it as its own publication. One integer for each pair therefore replaces the path list that both Usenet and Fidonet carried in every message. This is the reason no global identity is needed and it is the reason the path cannot be read.

A relationship gates injection. Every link costs a key exchange performed by two people who chose each other. This is the difference from Usenet that matters most. A server that publishes unwanted content at volume needs a peer, and a peer is a person who performed that exchange.

3. The model

This section gives the whole model before any encoding. Section 5 gives the encoding.

3.1 Items

A server holds items. An item is a piece of text with five fields:

Field	Content
handle	The public name of the writer at the server that wrote the item
topic	A dotted string, section 3.5
subject	One line of text
body	The text of the item
origin time	The time the item was created at the first server

An item is published or it is not published. Not published is the state at creation. A published item is offered to peers. A not published item is readable only on the server that holds it.

3.2 Pairs

Two servers are a pair when the administrator of each one approves the link.

The approval is a key exchange. Each side creates a key pair, and the two administrators exchange the public keys by any means outside the program. Each side then writes the public key of the other into its own records.

There is no registration, no directory, and no third party. A pair is independent of every other pair. A server that holds ten pairs holds ten key pairs, and no key is common to two of them.

3.3 Transfer

Transfer is pull. A target server asks an origin server for items above a cursor. The origin answers with items in ascending order. The target stores what it keeps and moves its cursor forward.

The cursor is one integer for each pair. Each side holds its own copy of it. This is the only state that the two sides share.

3.4 Re-origination

A target that publishes a received item offers it onward to its own peers. The item leaving the target carries a new sequence number from the counter of the target, and carries a flag

stating that the item is not local to the target.

A third server that pulls from the target receives the item as ordinary published content of the target. That third server learns that the item is not local to the target, and learns nothing else about where the item came from.

The result is that content moves any distance through a chain of pairs, and each hop is a normal publication. No path record travels with the item.

Figure 3.1 — One item across three servers

Server A	Server B	Server C
-----	-----	-----
writes item	pulls from A	pulls from B
publishes it	stores it	stores it
seq = 4193	seq = 118	seq = 62
remote = false	remote = true	remote = true
hash = H	hash = H	hash = H
	publishes it again	
	(relay = on)	
----- transfer ----->		
	----- transfer ----->	

The sequence number is new at every server.

The remote flag becomes true at the first hop and stays true.

The hash H is identical at all three servers and is never transmitted.

Nothing in the record names server A.

3.5 Topics

A topic is a lower case dotted string, for example `games.babylon5` or `alt.rock.and.roll`.

Nothing owns a topic. Nothing registers a topic. Nothing enforces agreement on a topic.

Two servers agree on a topic by using the same string.

Each server declares, for each pair, which topics it accepts from that pair and which topics it offers to that pair. The two declarations are independent and neither side sees the declaration of the other. An item passes only when it matches both.

The set of all such declarations is the routing of the network. There is no single graph.

Figure 3.2 — One graph for each topic

Servers: P Q R S T

Pairs: P-Q Q-R R-S Q-T

Topics carried on each pair:

```
      P --[games.*]-- Q --[games.*, music.*]-- R --
[music.*]-- S
```

```
      |
      |
      [music.*]
      |
      |
      T
```

Subgraph for games.* : P -- Q -- R

Subgraph for music.* : Q -- R -- S

```
      |
      |
      T
```

The two subgraphs use the same servers and the same pairs.

Server P holds no music item and spends nothing on music traffic.

Server S holds no games item and spends nothing on games traffic.

Servers P and S hold no link for any topic, although a path exists between them

3.6 The window

Each server holds a fixed count of received items and removes the oldest. The size of the store does not change once the window is full.

What changes with the volume that arrives is the length of time the window covers. A server that accepts many topics on a busy network holds a short history. A server that accepts one topic subtree holds a long history in that subtree.

The administrator sets the item count and the accepted topics. Those two settings together give the depth of history.

3.7 The model in one paragraph

A server holds items. An item is published or not. A published item is offered to peers. A pair is two administrators who exchanged keys. Transfer is pull, against a cursor. An item carries no path and no server identity. A target that publishes a received item offers it onward as its own publication. Each server holds a fixed window of received content and sets its own filters. Everything else is a local choice.

4. The three absent properties

This section is the argument of the paper. Each subsection states one absent property, states what its absence gives, and states what its absence costs.

4.1 No global identity of a writer

A handle exists on one server. It is 24 characters at most and it is the only name that travels with an item.

Two servers may hold a member with the handle `root`. Those are two different people. Nothing in the protocol detects this and nothing corrects it.

What this gives. A person is not addressable across the network. There is no global mention, no global follow, no global block, and no measure of reach that crosses a server. A person cannot accumulate an audience that is independent of a group, because the network carries writing and does not carry writers.

A second consequence is that no party holds a list of participants. There is no set of accounts anywhere. A server holds its own members and knows nothing about the members of any other server.

What this costs. Four capabilities are lost.

A block on a handle blocks every writer with that name behind the pair on which the block is set. Nothing detects the collision and nothing warns of it. An administrator who blocks a common handle should expect to lose unrelated writers with that name.

A reference to another item by handle, time, and subject is a description that a reader interprets. It is not a key that a program resolves. Two items may match one reference.

A reader cannot follow a writer. A reader follows a topic.

A writer who moves to another server is a new person to the network. Nothing connects the two handles.

4.2 No provenance

An item carries one boolean field, `remote`, that states whether the item is local to the server that is sending it. It carries no path, no hop count, no server identity, no member identity, and no signature of its own.

The mechanism that permits this is in section 3.4. Every hop is an ordinary local publication, so nothing needs to be recorded about the previous hops. Loop control uses a hash that each server computes for itself from the item fields and never transmits, described in section 5.8.

What this gives. The set of possible first servers for any received item is every server that the sending server can reach, directly or through further relays. A receiver cannot list that set and cannot narrow it.

A writer who is at risk from what they write is protected by the protocol at the level of the server. An item that reaches a hostile reader through four hops gives that reader no information about which of the servers behind the sender wrote it.

The absence of a per-item signature is part of this. A signature on an item would be verified against a key, and a key would identify a server. The signature in this design covers the transport only, between two servers that already know each other.

What this costs. A receiving administrator cannot separate the items of one first server from the items of another inside one pair. The controls therefore act on the pair, and section 7 gives them in full. The finest control that operates before storage is a list of blocked handles for one pair, which carries the collision cost of section 4.1.

A second cost is that the cost of an attack is constant while the cost of the defence rises with distance from the source. A server four hops from a flood can only cut the pair that fed it, and that pair carries legitimate content from a whole region of the network.

4.3 No directory

A server participates by speaking the protocol to a peer that agreed to speak it. There is no registry, no organisation, no name that anybody holds, and no version that anybody must adopt.

What this gives. Nothing can remove a server. There is no account to close, no listing to remove, no payment to stop, and no organisation to place under pressure. A server that every peer drops still runs, still holds its members, and still holds their writing. It rejoins the network by making one new link with one person.

A second consequence is that no party can observe the network. There is no place where the shape of the network is recorded. An administrator knows the pairs that administrator holds, and knows nothing else. An attacker who joins the network learns the same amount.

What this costs. Discovery has no mechanism. At a small server count an administrator knows the people they want to pair with. At a large one they do not.

The practical answer that the design permits is that an endpoint and a public key are published as an ordinary item under an agreed topic, relayed like any other item, and read by an administrator who then performs the key exchange out of band. Nothing in the protocol supports this, and nothing needs to. It is a convention among people and not a feature of the software.

A second cost is that a new server has no way to reach the network except through a person who already holds a link. The barrier to entry is a social one and it is real.

4.4 The three are one decision

The three properties are not independent. Removing one makes the removal of the others possible, and adding one back removes the others.

The dependency runs as follows.

A reply path carried **in the protocol** requires that a reply reach a specific parent item on a specific server. That requires a resolvable item identity, which requires a resolvable server identity, which is provenance. Provenance, once present in the record, identifies the first server. A resolvable author identity follows from the same requirement, because a reply is attributed to somebody.

Therefore: **the absence of a reply path in the protocol permits the absence of provenance, and the absence of provenance is what makes the first server unidentifiable.** A change that adds a global identity, a path field, a hop count, or a resolvable reference removes the other properties with it.

The one exception. A reply carried as **content** rather than as protocol does not break the chain. Section 6 describes a mechanism in which a reply quotes its parent in its own body, and every server recomputes the parent hash from that quote and finds its own copy. The quote carries the item fields only, and those fields already travelled with the parent. Nothing about the server that wrote the parent is added.

This exception is the reason that threading is possible in this design at all, and it is the reason section 6 exists as an optional part of the protocol rather than as a change to the model.

5. The wire protocol

5.0 Requirement keywords

In this section and in section 9, the words **must**, **must not**, and **may** have the meanings given in RFC 2119 and RFC 8174. In every other section of this paper the text is descriptive and no requirement is implied.

5.1 The endpoint

A server that serves content must provide one endpoint:

```
POST /fed/pull
```

```
Content-Type: application/json
```

No other path is defined and no other path is required.

TLS is expected on the outside. TLS gives privacy of the transfer. The signature of section 5.3 gives identity of the pair. They are separate and neither substitutes for the other.

A server that only pulls and never serves needs no endpoint. It is a conforming client.

5.2 The request

The body of the request is a JSON object with two fields.

```
{  
  "cursor": 4192,  
  "limit": 200  
}
```

Field	Type	Rule	Required
cursor	integer	The highest sequence the target holds from this pair. The first request must send 0.	Yes
limit	integer	The count of items the target accepts. The origin may send fewer. Must not be above 200.	Yes

An origin must reject a malformed body with status 400.

5.3 The signature

The target signs the request. The origin verifies it.

5.3.1 Keys

The algorithm is Ed25519, as defined in RFC 8032.

One key pair for each pair of servers, and not one for each server. A server with ten pairs holds ten private keys and ten public keys. No key is common to two pairs.

The public keys are exchanged out of band, by any means, between two people. There is no key server, no fingerprint registry, and no automatic exchange.

5.3.2 Headers

Two headers travel with the request:

X-Fed-Time: 1754870400

X-Fed-Sig: base64 (Ed25519 signature)

X-Fed-Time is Unix seconds in decimal ASCII, with no sign and no padding.

X-Fed-Sig is the signature encoded with standard base64.

5.3.3 The signed bytes

The signature input is the concatenation of four parts, with one zero byte between adjacent parts and no zero byte at the end:

Part	Content
1	The ASCII text POST
2	The ASCII text /fed/pull
3	The value of X-Fed-Time, decimal ASCII
4	The SHA-256 digest of the request body, as 32 raw bytes

Part 4 must be the raw bytes of the digest. It must not be a hexadecimal or a base64 encoding of them.

5.3.4 Verification

An origin must reject the request with status 401, and must write nothing, when any of the following holds:

- The value of X-Fed-Time differs from the clock of the origin by more than 300 seconds.
- The signature does not verify against the public key held for that pair.
- Either header is absent.

The clock is the most common cause of a total failure between two correct implementations. An operator should run a time synchronisation service.

5.4 The response

The body of the response is a JSON object with a list of items and two envelope fields.

```
{
  "items": [
    {
      "seq": 4193,
      "kind": "post",
      "handle": "root",
      "topic": "games.babylon5",
      "subject": "The shadow war",
      "body": "Text of the post.",
      "origin_time": 1754870400,
      "remote": false
    }
  ],
  "high": 4193,
  "more": false
}
```

5.4.1 Item fields

Table 5.1 — Item fields

Field	Type	Rule	Required
seq	integer	The sequence at the sending server, and at no other server	Yes
kind	string	One of post, reply, channel, message	Yes
handle	string	The name at the server that wrote the item. Not unique anywhere.	Yes
topic	string	Section 5.7. Null for reply and message.	Yes, may be null
subject	string	The subject or the channel name. Null for reply and message.	Yes, may be null
body	string	The text of the item	Yes
origin time	integer	Unix seconds, set at the first server, unchanged at every hop	Yes
remote	boolean	False at the first server, true at every hop after	Yes
parent seq	integer	For reply and message only. The seq of the parent at this sender.	Conditional

No other field is defined. A receiver must ignore a field it does not know. A sender must not add a field that identifies a server, a path, a hop count, or a member.

5.4.2 Envelope fields

Table 5.2 — Envelope fields

Field	Type	Rule
high	integer	The highest sequence the origin examined in this batch, and not the highest it sent
more	boolean	True when the origin holds items above high

high must be the examined value and not the sent value. The reason is that an outbound filter may remove every item in a batch. If high reported only what was sent, the cursor would not move and the origin would examine the same rows on every cycle. With the examined value, a pair with a narrow filter advances normally.

When more is true, the target may request again at once without waiting for its transfer interval.

5.5 The order

The origin must send items in ascending sequence order, lowest first.

A parent always holds a lower sequence than its child, because the parent is published first. A target therefore always holds the parent before the child arrives, within one sending server.

5.6 The sequence

The sequence is one integer for each server. It is the state that the transfer is measured against, and each side of a pair holds its own copy of the cursor.

Five rules must hold in any implementation:

1. A sequence value is assigned when an item becomes published.
2. The value must be higher than every value the server has issued before.
3. Withdrawal from publication must not release the value.
4. Republication must take a new and higher value.
5. **The sequence must never decrease.**

Rule 4 is the reason a publication counter is separate from any row identifier. A member may write an item today and publish it in a month. A row identifier assigned at creation would be below the cursor of every pair, and the item would never transfer. A publication counter assigned at publication is above the cursor, and the item transfers correctly.

Gaps in the sequence are harmless. The sequence is a high water mark and not a count.

Rule 5 can be broken only by a restore from a backup. After such a restore the administrator must raise the counter by hand above the highest value the server ever issued. Appendix C gives the procedure.

How the value is generated and stored is a local matter. A peer cannot observe it.

5.7 The topic grammar

A topic is a lower case dotted string. The rules are:

1. Two segments at least.
2. A dot separates two segments.
3. A segment starts with a letter, a to z.
4. A segment continues with a letter or a digit, a to z and 0 to 9.
5. No other character is valid.
6. No empty segment, no first dot, no last dot.
7. 128 characters at most.

Valid: `games.all, games.babylon5, alt.rock.and.roll, a.b`

Invalid: `games, games.5, .games.all, games.all., games..all, Games.All, games.all-x, games all`

A receiver must validate every arriving topic. A failure must discard the item and must advance the cursor.

The topic character set holds no glob character, so a filter pattern needs no escape rule.

The topic string is the only shared name in the system. Nothing enforces agreement on it and nothing needs to. Two servers agree on a topic by using the same string.

5.8 The hash

The hash is the duplicate key and the reply anchor. **Every implementation must produce the same bytes for the same item, or it is not conforming.**

The construction is the concatenation of five parts, with one zero byte between adjacent parts and no zero byte at the end, passed through SHA-256:

Part	Content
1	handle, UTF-8 bytes
2	topic, UTF-8 bytes
3	subject, UTF-8 bytes
4	body, UTF-8 bytes
5	origin time, decimal ASCII, no sign, no padding

Rules:

- A null field contributes zero bytes between its separators. The separators are still present.
- The topic must be hashed **as it arrived**, before any local prefix is applied.
- The hash must never be transmitted.

Each server computes the hash for itself. Because the input is item fields only, and because those fields do not change at a hop, every server computes the same value for the same item.

A receiver that already holds the value must discard the item and must advance the cursor. This is what stops a ring, and what stops one item arriving twice through two paths.

The rule that the topic is hashed before any prefix exists for a specific reason. A server may add a local prefix to an arriving topic. If the hash were computed after the prefix, two servers with different prefixes would compute different values for one item, and a ring passing through both would not be detected.

A known collision is accepted. Two members with the same handle on two servers, who write identical text at the same second under the same topic and subject, produce one hash. One copy is discarded. The alternative is a global item identity, which would carry the first server and would remove the property of section 4.2.

5.9 The filters

A pair holds two lists of glob patterns, one for each direction. Both lists may be empty.

Table 5.3 — Filter directions

Direction	Applied by	Applied to	Effect
out	The origin, when it builds a response	Items it would send	The origin decides what it offers this pair
in	The target, at receipt	Items that arrive	The target decides what it keeps

An item matches a list when it matches **any** pattern in that list. The list is a set of alternatives and not a sequence of rules, so the order of the patterns has no effect on the result.

An empty list passes nothing. This must be the default for a new pair. An administrator that wants everything writes one pattern:

*

The consequence is that a pair approved by accident carries nothing until the administrator

writes at least one pattern.

The two lists are independent. Neither side sees the list of the other. An item passes only when it matches both lists, so the effective set is the intersection. Each side can narrow without asking the other.

The outbound list is the more useful of the two because it saves transfer. The inbound list stays necessary as a defence, because a peer may not filter correctly.

Pattern rules

Item	Value
Character set	The topic set, plus *, ?, [,], -
Patterns in one list	64 at most
Length of one pattern	128 characters at most

Where the lists are stored is a local matter.

5.10 Receipt

The target must perform these steps, in this order, for each item in a response.

1. **Filter.** Test the topic against the `in` list of the pair. No match discards the item and advances the cursor.
2. **Validate.** Test the topic against the grammar of section 5.7. Test every field against the limits of section 5.12. A failure discards the item and advances the cursor.
3. **Hash.** Compute the hash of section 5.8 from the fields as they arrived.
4. **Prefix.** Apply a local topic prefix if the pair holds one. This step is local and is after the hash.
5. **Store.** A hash conflict discards the item and advances the cursor.
6. **Map.** Record the relation between the `seq` of the sender and the local item, so that a later `parent_seq` can be resolved.
7. **Cursor.** Advance the cursor to the `seq` of the item.

Steps 5, 6, and 7 must be one atomic unit. A failure in the middle of a batch must leave the cursor at the last committed item, and the next transfer must resume there.

Every discard must advance the cursor. A discarded item must never be requested again.

Figure 5.1 — The receipt sequence

item arrives

```

|
v
[1] topic in the "in" list? --no--> discard, advance cursor
| yes
v
[2] topic grammar valid?    --no--> discard, advance cursor
field lengths valid?      --no--> discard, advance cursor
| yes
v
[3] compute hash from the fields as they arrived
|
v
[4] apply local prefix, if any      (local, after the hash)
|
v
[5] store -- hash already present? --yes--> discard, advance cursor
| no                                |
v                                |
[6] record the seq to local relation      |
|                                |
v                                |
[7] advance cursor                      |
|                                |

```

+---- steps 5,6,7 are one atomic unit -----+

5.11 Relay

Relay is a property of the pair, decided once when the link is made. It is not a decision made for each item, and no human step exists in the path of an item.

Relay	Effect at receipt
Off	The item is stored unpublished. It is readable on this server and goes no further.
On	The item is published on arrival with a new local sequence. It goes to every other pair on the next cycle.

The default must be off.

An item leaving a relaying server must carry `remote` set to true and a new `seq` from that server. The next server learns that the item is not local to the sender, and learns nothing else.

A published item cannot be recalled. The first server holds no address of any copy. A delete is local to one server. An implementation must not present a control that claims otherwise.

5.12 Limits

Table 5.4 — Field and transfer limits

Item	Value
Topic	128 characters
Subject, channel name	200 characters
Post body	16 KB
Reply body	4 KB
Chat message	2 KB
Handle	24 characters
Items in one response	200
Response body	4 MB
Signature age	300 seconds
Transfer interval	60 seconds at least
Patterns in one filter list	64
Length of one pattern	128 characters

A server must reject a response above the response body limit. A server must discard an

item above a field limit and must advance the cursor.

6. Replies

6.0 Status

This section is optional. A server that does not implement it stores replies as ordinary items and interoperates without fault.

The only matter on the wire is the format of the quote block in section 6.2, because both sides must parse the same fields out of it to compute the same parent hash.

6.1 The principle

The hash of section 5.8 is computed from item fields only, and every server computes the same value for the same item. It is already the duplicate key.

A reply quotes the fields of its parent in its own body. Those fields are exactly the fields that already travelled with the parent. A receiving server therefore recomputes the parent hash from the quote and finds the parent in its own store with one lookup.

The identity of the parent is its content. Nothing else is needed, and nothing else travels.

The quote block is also the display of the parent for a reader whose server does not hold it, so the mechanism costs nothing beyond what a quoted reply already contains.

6.2 The quote block

The block is the first part of the body. Each line starts with a > character and one space. The first line is the header. An empty line ends the block. The reply text follows.

```
> handle @ decimal(origin_time) | topic | subject
```

```
> The body of the parent.
```

```
> Further lines of the parent body.
```

The text of the reply.

The five fields parsed from the block are the five hash inputs, in the order of section 5.8:

Field	Source in the block
handle	Header, before the @
origin time	Header, between the @ and the first

Field	Source in the block
topic	Header, between the first and the second
subject	Header, after the second
body	Every line after the header, with the > prefix removed and the final newline discarded

A quoted body above the limit of section 5.12 is truncated by the writer. A truncated quote does not produce the parent hash, and the reply is then treated as an item with no resident parent. An implementation that truncates should mark the quote as partial for the reader.

The block is text. A reader on a server that does not implement this section sees a quoted reply and loses nothing.

6.3 A reply is a post

A reply written on a server is an ordinary local item. It takes a topic, usually the topic of its parent. It takes a sequence when published. It transfers to every pair whose outbound list matches the topic. It relays under section 5.11 in the ordinary way.

It is filtered, hashed, relayed, trimmed, and loop controlled by the rules of section 5 with no exception.

6.4 Resolution at receipt

Where an implementation supports this section, the following is inserted between step 5 and step 6 of section 5.10, inside the same atomic unit.

1. Parse the quote block. No block gives an ordinary top level item. Stop.
2. Compute the parent hash from the parsed fields, by the rule of section 5.8, using the topic as quoted and before any local prefix.
3. Look for a local item whose hash equals that value. A match sets the local parent relation.
4. No match gives an item with no resident parent. The server keeps it or discards it, by local setting.

The same procedure runs when a member writes a reply locally.

6.5 Late attachment

Order is not guaranteed across pairs, so a reply may arrive before its parent.

On every insert of any item, after the hash is computed, the server sets the parent relation on every stored item whose parent hash equals the hash of the item being inserted.

An item with no resident parent therefore binds at the moment its parent arrives. A server that receives a conversation in reverse order ends with the same tree as a server that received it in sequence.

Figure 6.1 — Late attachment

time ->

t1 reply R arrives. Its parent hash is H.

 No local item has hash H.

 R is stored with no parent relation.

t2 item P arrives from another pair. Its hash is H.

 P is stored.

 The insert of P sets the parent relation on every stored
 item whose parent hash is H.

 R now points at P.

 The tree is the same as if P had arrived first.

 No queue, no retry, and no timer is involved.

6.6 The window bounds the mechanism

A reply attaches only to a parent that the server still holds. Past the window the parent is gone, the lookup fails, and the item stands alone.

This bounds the mechanism with no rule of its own:

- The search set is the resident window and never more.
- A thread ages out whole, because the parent and the replies trim on the same schedule.
- No structure accumulates. There is no thread table, no queue of unattached items, and no tail.

The depth of threading follows the depth of history, which follows the narrowness of the inbound filter. The lever is the one the administrator already has.

6.7 Convergence

Two servers holding one parent may hold different sets of replies. Both trees are correct for their position.

Condition	Result
Both servers accept the topic and are connected within its subgraph	The trees converge within a few transfer intervals
One server has a narrower inbound list	It holds the subset that matched
One server has a shallower window	It holds the subset still resident
The subgraph is partitioned	Two trees, neither aware of the other

There is no thread state, no agreement procedure, and no reconciliation. A tree is the local result of what arrived.

6.8 The return path

Server A publishes an item. Server B pulls it and a member of B writes a reply. The reply is a published item of B under the topic of the parent.

When A pulls from B, A computes the parent hash from the quote, finds its own item, and attaches the reply to its own tree.

A reply returns when a pair runs in that direction and both servers carry the topic. No push exists, no address is held in the item, and no server learns anything it did not already hold.

The reply also reaches every other server that carries the topic and holds the parent, by ordinary relay, without passing through A. Such a server attaches the reply whether or not it holds any link to A.

Nothing confirms delivery and nothing reports back to the writer of the reply.

6.9 Why this does not add provenance

The quote carries the item fields only, and those fields already travelled with the parent. A reader sees the handle and the time of the parent, which they would see from the parent itself. Nothing about the server that wrote the parent is present in the quote, because nothing about it was present in the parent.

The hash is computed locally and never transmitted. It is not an identifier that resolves. It is a value that two servers happen to agree on because they computed it from the same input.

This is the exception described in section 4.4. The reply is content and not protocol, so no capability of section 4.4 is reintroduced.

6.10 The weaker form — optional

A second and weaker mechanism exists. It is described here for completeness and an implementation may ignore it entirely.

An item may carry one text field that names another item, in the form `handle:timestamp:subject`, with the split taken at the first two colons so that a subject holding a colon stays whole. The three parts are the three fields that travel unchanged to every server, so the reference means the same thing everywhere.

Nothing resolves it. No server checks that the named item exists, and no server can. A malformed value is discarded and the item that carries it is kept. The field is not part of the hash input of section 5.8.

The mechanism gives a reader a way to see the responses that reached that server. It does not give the set of responses that exist, and an interface must not suggest that the list is complete.

7. Control and its limits

7.1 The controls

Table 7.1 — Controls, from coarse to fine

Control	Scope	Collateral
Drop the pair	Every item ever received through it	Total for that link
Disable the pair	Future items of the pair	Holds what is already stored
Inbound topic list	Topic subtrees of the pair	Whole subtrees
Inbound handle block list	Named handles of the pair	Every writer with those names behind that pair
Delete an item	One item	None

An outbound handle block list also exists. It prevents named local members from reaching one specific peer, while those members still reach every other peer.

7.2 Control is a decision made before the fact

An administrator makes three decisions once, when a pair is made:

- Which topics come in
- Which topics go out
- Whether items from this pair relay onward

Content then passes automatically inside those choices. There is no moderation queue, no report flow, no appeal, and no policy document.

This is a consequence of section 4.2. Provenance does not travel, so no control can be conditioned on where an item came from. The only thing an administrator can condition on is the pair, the topic, and the handle.

7.3 A relaying server will relay something its administrator would not have chosen

This follows directly from section 7.2 and this paper states it plainly.

A server with relay enabled on a pair carries every item that passes the filters, with no human step. Over time such a server will have relayed content that its administrator, if asked in advance, would have refused.

No control exists after the fact. The copies that left are beyond reach. The controls are the pair and the filter, applied before the fact.

An administrator who cannot accept this should not enable relay. A server with relay disabled on every pair is fully conforming, reads everything its filters accept, and passes nothing onward.

7.4 A volume attack clears itself

A hostile server publishes at volume. The items reach every server in the subgraph that accepts the topic.

Three properties bound the damage.

The filter bounds the reach. Content under one topic does not touch a server that accepts a different topic. The volume is confined to the subgraph that asked for that topic.

The window clears it. The items age out of every store at the same rate as everything else. There is no cleanup operation and no administrator action is required for the content to leave.

The lasting cost is history and not storage. What is lost is the window depth of the servers that received the items, for the period the attack ran. The store size never changes.

7.5 The asymmetry that remains

The cost of the attack is constant. The cost of the defence rises with distance from the source.

A server adjacent to the source cuts one pair and the problem stops. A server four hops away can only cut the pair that fed it, and that pair carries legitimate content from a whole region of the network.

This is the direct cost of section 4.2 and it is not mitigated. The mitigations that exist are the filter, the window, and the pairing requirement itself.

7.6 The pairing is the rate limiter

Every link costs one key exchange performed by two people who chose each other. Nothing technical bounds the growth of the network.

This is the governor of the design. It is slow, and it is chosen because it is slow. A party that wishes to inject content at volume needs a peer, and a peer is a person who agreed.

8. Scale

All figures in this section are estimates derived from the design and from the field limits of section 5.12. None is a measurement. No large deployment of this protocol exists at the time of writing.

8.1 The store is a window

Every server holds a fixed count of received items and removes the oldest.

There is no growth curve. The store reaches its steady state at the moment the window fills and stays there. The count does not depend on the count of servers, the count of pairs, the age of the network, or the volume of the reachable set.

The quantity that changes with the volume of the network is the wall clock time the window covers. A busier network gives a shorter history at the same item count.

Table 8.1 — Window depth against filter width (estimate)

Accepted topics	Window at 100,000 items
Everything	Hours to days
One second level subtree	Weeks
One leaf topic	Months to years

A narrow filter buys depth of history. This is the trade an administrator makes, and it suits a group with a specific interest: deep history in the topic that group reads, and no history at all in every other topic.

8.2 Storage

A typical item is about 1 KB. The field limit for a post body is 16 KB and for a chat message is 2 KB.

100,000 items at 1 KB is about 100 MB. 1,000,000 items at 1 KB is about 1 GB.

A server holding 100,000 items holds a quantity of data that fits in the memory of a small instance. Storage is not a constraint of this design at any server count the design is likely

to reach. A specification of the store belongs in the window count and not in a capacity plan.

8.3 Bandwidth

The content is text. A server sends each item once for each pair.

Table 8.2 — Egress estimates

Case	Egress
3 pairs, 1,000 items each day	About 3 MB each day
20 pairs, 10,000 items each day	About 200 MB each day
50 pairs, 10,000 items each day	About 500 MB each day

The last figure averages below 50 kbit/s. These quantities are inside the included allowance of a small instance from any provider.

Bandwidth does not enter the design of this system. An analysis that treats the multiplication by pair count as a limit has applied a model built for media to a system that carries text.

8.4 Propagation

Propagation time is the sum of the transfer intervals along a path, plus up to one interval of phase offset at each hop.

Table 8.3 — Propagation estimates at a 60 second interval

Servers	Diameter within a topic subgraph	Time
100	3 to 4 hops	3 to 8 minutes
1,000	4 to 5 hops	4 to 10 minutes

The hop counts above are an estimate and this paper does not cite a source for them. The estimate assumes that the graph has the clustering and the path length typical of a graph that grows from existing relationships between people, because each pair in this design is exactly such a relationship. A deployment may differ and no measurement exists.

The graph is not designed. It grows the way relationships between administrators form. The routing behaviour is a consequence of that structure and not of any routing decision.

8.5 What changes with scale

Three things change. None is a resource.

The topic namespace becomes contested. The topic string is the only shared name and

it determines routing. Two groups using different strings for one subject are two networks. Two groups using one string for different subjects collide in every filter that accepts it. At a small server count this is settled by conversation between administrators. At a large one, competing strings for one subject are permanent.

A local topic prefix is the tool that bridges them. One server maps an arriving topic onto another string at receipt. This joins two topic communities without the agreement of either, by the decision of one administrator.

Pairing is the only rate limiter. Each link costs one key exchange performed by a person. The growth rate of the network is bounded by the rate at which relationships between administrators form, and by nothing technical.

Discovery has no mechanism. Section 4.3 states the cost and the convention that answers it.

8.6 The shape at scale

Table 8.4 — Summary, 100 servers against 1,000 servers

Property	100 servers	1,000 servers
Store for each server	Fixed by the window	Identical
Bandwidth	Trivial	Trivial
Hardware cost of a relay	Near zero	Near zero
Diameter within a topic	3 to 4 hops	4 to 5 hops
Propagation	3 to 8 minutes	4 to 10 minutes
Window depth, broad filter	Longer	Shorter
Window depth, narrow filter	Unchanged	Unchanged
Effective topology	One graph for each topic	More graphs, and each more selective
Volume attack	Clears itself	Clears itself
Growth governor	Human pairing	Human pairing

Nothing technical changes between the two columns. The window depth for a server with a broad filter shortens, which moves administrators toward narrower filters, which separates the network into topic subgraphs.

That separation is the design working. A network of a thousand servers is not one large network. It is several hundred small ones sharing infrastructure, which is the correct shape for a federation of independent groups.

9. Conformance

9.1 Requirements

An implementation is conforming when all eight of the following hold.

#	Requirement	Reason
1	It produces and accepts the request of section 5.2 and the response of section 5.4	The wire format
2	It sends items in ascending sequence order	A parent must arrive before its child
3	It computes the hash of section 5.8 byte for byte	Loop control and reply anchoring depend on agreement
4	It computes the signature input of section 5.3.3 byte for byte	Authentication of the pair
5	It validates topics by section 5.7	Filters and routing depend on a fixed grammar
6	It advances the cursor on every item, including a discard	A discarded item must never be requested again
7	It never decreases its sequence	A cursor that is above the counter receives nothing
8	It adds no field that identifies a server, a path, a hop count, or a member	The properties of section 4

Nothing else is required. Every other choice is local.

9.2 The two constructions that must be exact

The hash and the signature input are the only two byte level agreements in the protocol.

A signature mismatch fails loudly. The origin returns 401 and the administrator sees it.

A hash mismatch fails silently. Rings stop being detected, duplicates multiply through the network, and replies do not thread. Nothing reports an error and nothing looks wrong on either side until the duplicates accumulate.

An implementer must test the hash against the vectors of Appendix A before pairing with anybody.

9.3 What is local

A peer cannot observe any of the following and has no reason to want to.

Local matter	Note
The storage system	A relational database, a file, or memory
Every internal name	No column name or field name is on the wire
The transfer loop	A thread, a scheduled job, or a person running a command by hand
The window count and the trim	Each server sets its own depth of history
Where the filters are held	A table, a configuration file, or a fixed list
Threading	A server may implement section 6 or ignore it
The interface	Web, terminal, a mail gateway, or none
Member accounts	A server may have zero members and only relay

An implementation may be a full server, a relay with no members, a read only client that pulls into a local file, a deep archive of one topic, or a gateway to some other medium entirely. A peer cannot distinguish them.

10. Properties and limits

This section collects every property that a reader must know before implementing or running a server.

Table 10.1 — Properties

#	Property
1	Publication is final. A copy at a peer stays there. The first server holds no address of any copy.
2	A handle is not unique. Two members with one name on two servers are two members, and nothing detects this.
3	The pair is the coarse unit of control. A handle block list is the finest control that operates before storage, and it carries the collision cost of property 2.
4	The topic string is the only shared name. Nothing enforces agreement on it.

#	Property
5	A ring is safe. The hash stops the loop, at the cost of one lookup for each item.
6	A reply returns only when a pair runs in that direction and both servers carry the topic. Nothing confirms delivery and nothing reports back.
7	A reader sees the responses that reached that server, and not the set that exists. There is no way to obtain that set.
8	A correction does not follow the error. A correction is a new item and travels a different path.
9	The store is a window. History is lost by design at the rate the window fills, and only a storage snapshot recovers it.
10	Discovery has no mechanism in the protocol.
11	A relaying server will at some point have relayed content its administrator would not have chosen. No control exists after the fact.
12	The cost of a defence rises with distance from the source of a problem.

10.1 The residual risk that is not technical

Only the handle travels, and the handle means nothing outside one server. The protocol therefore protects the identity of the server that wrote an item.

It does not protect the identity of the person who wrote it. A body written by a person identifies that person by style, by subject matter, and by reference to events that are local to that person. An analysis of the text can achieve what an analysis of the protocol cannot.

This paper states this plainly and does not soften it. An implementation must not present the protocol as protection for a writer against analysis of what that writer wrote. The protocol protects the server. That is the whole of the claim.

11. Conclusion

This paper has described a protocol for text distribution in which three properties common to prior and current federated systems are absent. A writer has no global identity. An item carries no provenance. A server appears in no directory. In place of all three, one link

between two servers is a relationship between two named people who exchanged keys outside the program, and every other decision is local to one server or to one pair.

The design produces a set of small groups. Each group is closed and holds its own membership and its own fixed window of content. The groups are connected by links that people made deliberately. The content carries names that nobody owns, under topic strings that nobody registers, with no way to determine where anything started and no way to take anything back.

The governor of the network is the key exchange. Each link costs one exchange between two people, and nothing technical bounds the growth of the network. That is a slow mechanism, and it is chosen because it is slow. Section 7.6 gives the reason: a party that wishes to inject content at volume needs a peer, and a peer is a person who agreed.

There is no reference implementation that holds authority, no organisation, no registry, no name that anybody holds, and no version that anybody must adopt. A server participates by speaking this protocol to a peer that agreed to speak it. Nothing else grants participation and nothing can withdraw it. Anyone may implement this. Anyone may run it. Nobody needs permission, and nobody can be removed. The protocol survives every implementation of it, including the first one.

Appendix A — Test vectors

Status of this appendix: normative. An implementation must reproduce every value here.

A.1 Hash, all fields present

Input fields:

```
handle:      root
topic:       games.babylon5
subject:     The shadow war
body:        Text of the post.
origin_time: 1754870400
```

Input bytes to SHA-256, with \x00 marking one zero byte:

```
root\x00games.babylon5\x00The shadow war\x00Text of the post.\x001754870400
```

Length of the input: 78 bytes.

Expected digest, hexadecimal:

[TO BE COMPUTED AND INSERTED BEFORE RELEASE]

A.2 Hash, null fields, as in a reply

Input fields:

```
handle:      root
topic:       (null)
subject:     (null)
body:        A reply.
origin_time: 1754870400
```

Input bytes to SHA-256:

```
root\x00\x00\x00A reply.\x001754870400
```

Length of the input: 25 bytes.

Expected digest, hexadecimal:

[TO BE COMPUTED AND INSERTED BEFORE RELEASE]

Note that the separators for the two null fields are present and the fields contribute no bytes. An implementation that omits the separators for a null field produces a different value and is not conforming.

A.3 Signature input

For a request body B sent at time 1754870400, the bytes passed to the Ed25519 signing function are:

```
POST\x00/fed/pull\x001754870400\x00<32 raw bytes of SHA-256(B)>
```

Length of the input: $4 + 1 + 9 + 1 + 10 + 1 + 32 = 58$ bytes.

The final 32 bytes are the raw digest. An implementation that inserts a hexadecimal or a base64 encoding produces a 122 byte or a 102 byte input and will not verify against a conforming peer.

A.4 Topic validator

Table A.1 — Topic validator cases

Input	Result	Rule
games.all	Valid	
games.babylon5	Valid	A digit after a letter
a.b	Valid	The shortest valid form
alt.rock.and.roll	Valid	Four segments
games	Invalid	One segment
games.5	Invalid	A segment starts with a digit
.games.all	Invalid	A first dot
games.all.	Invalid	A last dot
games..all	Invalid	An empty segment
Games.All	Invalid	Upper case
games.all-x	Invalid	A character outside the set
games all	Invalid	A space
(empty)	Invalid	Zero length
.	Invalid	No segment
129 characters	Invalid	Above the length limit

A.5 Note to the reader of this draft

The two digest values in A.1 and A.2 are not present in this draft. **A test vector without its expected output is not a test vector.** These two values must be computed from the byte sequences given above and inserted before this paper is released. This is the single most important open item in the document.

Appendix B — Field travel

Status of this appendix: informative.

Table B.1 — Which values leave a server

Field	Leaves	Changes at a hop	Note
handle	Yes	No	The name at the first server. Not unique.
topic	Yes	Only by a local prefix	Section 5.10 step 4
subject	Yes	No	
body	Yes	No	

Field	Leaves	Changes at a hop	Note
origin time	Yes	No	Set at the first server, held to the last
remote	Yes	Yes	False at the first server, true at every hop after
seq	Yes	Yes	The number of the sending server only
parent seq	Yes	Yes	Resolved through the local map of section 5.10 step 6
The hash	No	—	Computed at each server. Identical everywhere.
The pair identifier	No	—	A local label
The publication counter	Only as seq	—	
A member email address	No	—	No page shows it and no transfer carries it
Invitation lineage	No	—	Local to the server
Server identity	No	—	Nothing carries it
The public key	No	—	Used in the transport, not in the record
Hop count	No	—	Not held and not derivable

The last four rows constitute the anonymity of the first server. **Every one must stay absent for the property of section 4.2 to hold.** A change that adds any of them to the record removes the property.

Appendix C — Failure modes

Status of this appendix: informative.

Table C.1 — Failures, detection, effect, and action

Event	Detection	Effect	Action
The origin is offline	Connection failure	The cursor holds. Nothing is lost.	Retry with a growing wait
The origin changes its address	Connection failure	Transfer stops	The administrator edits the endpoint
The clock differs by more than 300 seconds	401 at the origin	All transfer stops for that pair	Run a time synchronisation service. This is the most common cause of a total failure.
The signature fails	401 at the origin	No transfer and no write	Verify the keys and the signature input construction
The hash is computed differently by two implementations	Nothing	Rings are not detected, duplicates accumulate, replies do not thread	Test against Appendix A. This failure is silent.
The origin is restored from a backup	The cursor of the target is above the counter of the origin. The target receives nothing further.	Transfer appears to stop with no error	The origin raises its counter above the highest value it ever issued
The target loses its cursor, value too low	Items arrive a second time	None. The hash rejects them.	None needed. The cost is transfer.
The target loses its cursor, value too high	Items are skipped with no error	Content is missing	Set the cursor to 0. Every item arrives again and the hash keeps one copy of each.
An item is above a field limit	The length test at receipt	The item is discarded, the cursor advances	None. The loss is one item.
An arriving topic is invalid	The grammar test at receipt	The item is discarded, the cursor advances	None
A ring returns an item	A hash conflict	The item is discarded, the cursor advances	None. This is the intended behaviour.
Two servers use one handle	Nothing	Two writers appear under one name	Nothing at the protocol level

Event	Detection	Effect	Action
A relaying pair carries unwanted content onward	A report from a downstream server	Copies exist beyond reach	Disable relay on the pair, or drop the pair. Past copies stay.
The storage device is full	An insert failure	The transaction rolls back and the cursor holds	Free space, or reduce the window count

Setting a cursor to 0 is always safe and always correct. Every item arrives again and the hash keeps one copy of each. The cost is one full transfer. This is the general repair for any doubt about a pair.

The publication counter must never decrease. A restore from a backup is the only event that can break this, and an administrator must raise the counter by hand after such a restore.

Appendix D — Deployment note

Status of this appendix: informative. Nothing here is required for conformance.

D.1 Storage

The storage requirement follows from section 8.2 and is small. A server holding 100,000 received items at a typical size of 1 KB holds about 100 MB.

An implementation that uses an embedded database file should verify that the storage device provides correct file locking. Network file systems and object storage accessed through a file system layer commonly do not, and the failure mode is data corruption rather than a visible error.

D.2 Making a pair

1. The administrator adds a record with a label and the endpoint of the other server.
2. The implementation creates the key pair and displays the public key.
3. The two administrators exchange the public keys outside the program.
4. Each administrator writes the key of the other into the record.
5. Each administrator writes at least one pattern into the filter lists. An empty list carries nothing.
6. Each administrator enables the pair.

Step 5 is the step most often forgotten. A pair with empty filter lists is enabled, authenticates correctly, transfers correctly, and carries no items. This is the intended behaviour of section 5.9 and it is not a failure.

D.3 The transfer loop

An implementation may run one loop for each enabled pair, or one loop that visits every pair, or no loop at all if an administrator runs a transfer by hand.

A failure should be followed by a wait that grows, to a bound of about one hour. A success should reset the wait.

The transfer interval must be 60 seconds at least, by section 5.12.

D.4 What to verify before pairing with anybody

1. The two hash vectors of Appendix A.
2. The signature input length of Appendix A.3.
3. The fifteen topic validator cases of Appendix A.4.
4. That a discarded item advances the cursor.
5. That the sequence never decreases across a restart.

Items 1 and 4 are the two that fail silently.

References

#	Reference
1	RFC 5536, <i>Netnews Article Format</i> . For the Path header and the Message-ID.
2	RFC 3977, <i>Network News Transfer Protocol</i> . For the pull transfer model.
3	FTS-0001, <i>A Basic Fidonet Technical Standard</i> . For SEEN-BY, PATH, and the nodelist.
4	<i>ActivityPub</i> , W3C Recommendation, 2018. For the actor URI and the push model.
5	RFC 8032, <i>Edwards-Curve Digital Signature Algorithm (EdDSA)</i> . For Ed25519.
6	FIPS 180-4, <i>Secure Hash Standard</i> . For SHA-256.

#	Reference
7	RFC 2119 and RFC 8174. For the requirement keywords.

Note on two absent references

Two claims in this paper are not supported by a citation.

Section 2.2 describes the mechanism most often given for the decline of the open Usenet text hierarchies. The paper marks this as a design input and not as an established finding. A citable account should be found, or the paragraph should be reduced further.

Section 8.4 gives hop counts for networks of 100 and 1,000 servers. The paper marks these as estimates and states the assumption they rest on. They should either be supported by a citation on the diameter of graphs that grow from relationships between people, or left as marked estimates.

Open items before release

#	Item	Status
1	Compute and insert the two hash digests in Appendix A.1 and A.2	Required. Blocking.
2	Find a citable source for the Usenet claim in section 2.2, or reduce it	Open
3	Support or leave marked the hop count estimates in section 8.4	Marked as an estimate. Acceptable as it stands.
4	Choose a licence for the protocol text. Section 11 requires it to place no restriction on implementation.	Open
5	Decide whether to state that a reference implementation exists, and that it holds no authority	Open
6	Verify the byte lengths stated in Appendix A.1, A.2, and A.3 against a real implementation	Open

End of paper.

[[HOWL-COMP-15-2026](https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-15-2026)] Human Trust Based Federated Publication. Github:
<https://github.com/ghowland/papers/tree/main/papers/COMP/HOWL-COMP-15-2026>